

Problems

```
import pandas as pd
import numpy as np
import sklearn
import matplotlib.pyplot as plt
```

Scoring

- Problem 1 is worth 10 points;
- Problem 2 is worth 15 points;

Problem 1

Let's examine the influence of global temperature change on the blooming day of cherry blossoms in Kyoto. While these dates are discrete, we will use a linear regression model relating the day to the global mean temperature anomaly for March (the third non-year column in `data/HadCRUT5.1Analysis_gl.txt`).

Problem 1.1

Load the data and fit this model. What does your fitted model suggest as the influence of March temperature anomalies on the cherry blossom date?

```
blossoms = pd.read_csv("data/kyoto.csv")
blossoms = blossoms[blossoms["Year.AD"] > 1850].reset_index(drop=True)
```

```
years_na_blossoms = np.where(blossoms[' Flowering.DOY'].isna())[0]
blossoms = blossoms.drop(years_na_blossoms).reset_index(drop=True)
```

```
temp_dat = pd.read_csv("data/HadCRUT5.1Analysis_gl.txt", sep="\s+",
    names=["Year", "January", "February", "March", "April", "May", "June", "July", "August",
stop = temp_dat.index[temp_dat["Year"]==2015][1]
temps = temp_dat[:stop:2].reset_index(drop=True)
temps = temps[1:].reset_index(drop=True)
temps = temps.drop(years_na_blooms).reset_index(drop=True)

model = sklearn.linear_model.LinearRegression()
model.fit(pd.DataFrame(temps["March"]), pd.DataFrame(blossoms[" Flowering.DOY"]))
```

fit_intercept fit_intercept: True
bool, default=True

Whether to calculate the
intercept for this model. If
set
to False, no intercept will be
used in calculations
(i.e. data is expected to be
centered).

copy_X copy_X: bool, True
default=True

If True, X will be copied;
else, it may be overwritten.

tol tol: float, default=1e-6 1e-06

The precision of the solution ('coef_') is determined by 'tol' which specifies a different convergence criterion for the 'lsqr' solver. 'tol' is set as 'atol' and 'btol' of :func:'scipy.sparse.linalg.lsqr' when fitting on sparse training data. This parameter has no effect when fitting on dense data.

.. versionadded:: 1.7
n_jobs n_jobs: int, None
default=None

The number of jobs to use for the computation. This will only provide speedup in case of sufficiently large problems, that is if firstly 'n_targets > 1' and secondly 'X' is sparse or if 'positive' is set to 'True'. "None" means 1 unless in a :obj:'joblib.parallel_backend' context. "-1" means using all processors. See :term:'Glossary' for more details.

positive positive: bool, False
default=False

When set to “True“, forces the coefficients to be positive. This option is only supported for dense arrays.

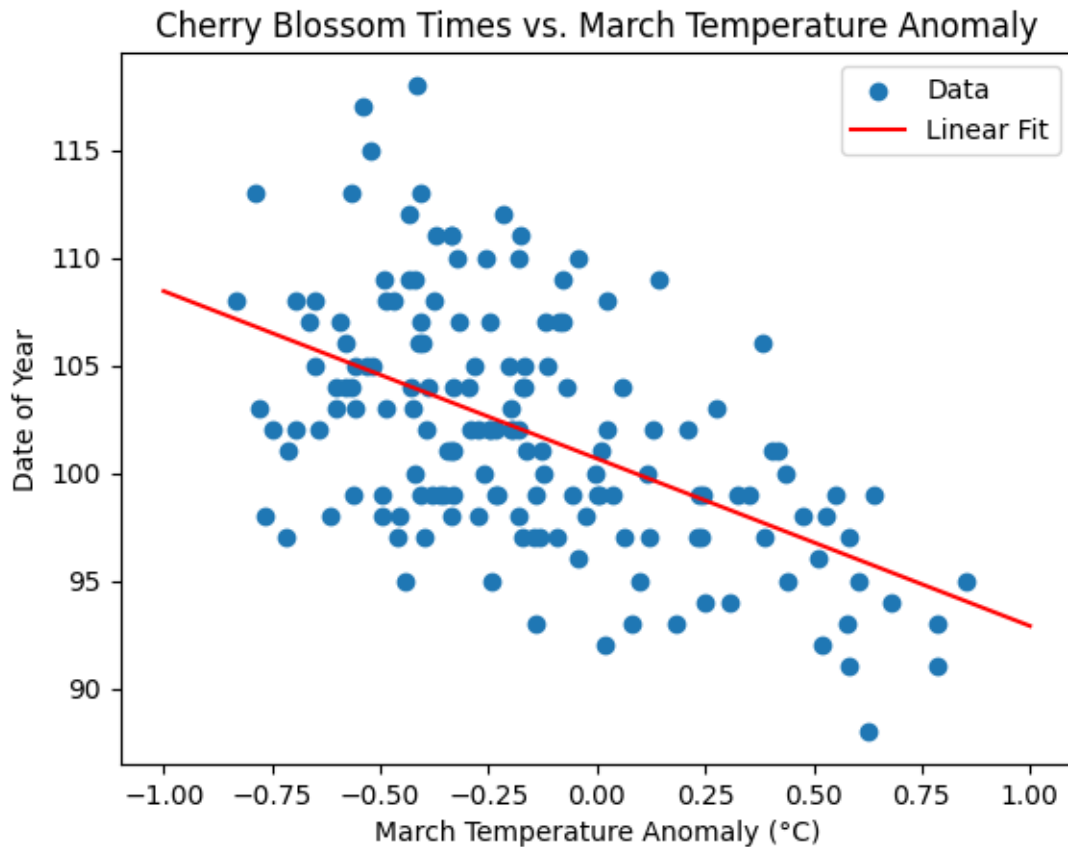
For a comparison between a linear regression model with positive constraints on the regression coefficients and a linear regression without such constraints, see [:ref:‘sphinx_glr_auto_examples_linear_model_plot_nnls.py’](#).

.. versionadded:: 0.24

```
model.coef_[0][0]
```

```
np.float64(-7.773816956010837)
```

```
plt.scatter(temps["March"], blossoms[" Flowering.DOY"], label="Data")
x = np.linspace(-1, 1, 100)
plt.plot(x, (model.coef_[0][0]*x)+model.intercept_[0], "r-", label="Linear Fit")
plt.legend()
plt.xlabel("March Temperature Anomaly (°C)")
plt.ylabel("Date of Year")
plt.title("Cherry Blossom Times vs. March Temperature Anomaly")
plt.savefig("figs/11.png")
```



Problem 1.2

Calculate the log-loss (negative log-probability) and mean-squared error metrics for the fitted model on the in-sample data.

```
sklearn.metrics.mean_squared_error(pd.DataFrame(blossoms[" Flowering.DOY"]), model.predict(p
```

22.316707714914653

Problem 1.3

Conduct a 5-fold cross-validation for the model using both of these metrics. How do they compare to the in-sample metrics?

```
sklearn.model_selection.cross_val_score(sklearn.linear_model.LinearRegression(),
    pd.DataFrame(temps["March"]),
    pd.DataFrame(blossoms[" Flowering.DOY"]),
    cv=5,
    scoring="neg_mean_squared_error")
```

```
array([-34.675388 , -23.14703813, -20.46070791, -22.81506239,
       -18.74007898])
```

Problem 2

Let's re-examine the influence of air pollution and temperature on mortality in Chicago, IL. The `data/chicago.csv` dataset (originally from the [gamair R package](#)) contains data on the relationship between environmental conditions (temperature and air quality) and deaths in Chicago from 1987–2000. The variables are:

- the total number of (non-accidental) deaths each day (`death`);
- the median density over the city of large pollutant particles (PM10; `pm10median`);
- the median density of smaller pollutant particles (PM2.5; `pm25median`);
- the median concentration of ozone (O₃) in the air (`o3median`); the median concentration of sulfur dioxide (SO₂) in the air (`so2median`);
- the time in days (`time`); and the daily mean temperature (`tmpd`).

The pollution variables have been standardized to represent deviations from the overall median and the temperature is in Fahrenheit. The days are given as a number of days relative to December 31, 1993. Note that some values are missing; these can be treated as missing at random and dropped in your regressions.

```
chi = pd.read_csv("data/chicago.csv", index_col="idx")
```

```
chi["date"] = pd.to_datetime("1993-12-31") + pd.to_timedelta(chi.time, "day")
```

```
chi
```

	death	pm10median	pm25median	o3median	so2median	time	tmpd	date
idx								
1	130	-7.433544	NaN	-19.592338	1.928043	-2556.5	31.5	1986-12-31 12:00:00
2	150	NaN	NaN	-19.038614	-0.985563	-2555.5	33.0	1987-01-01 12:00:00
3	101	-0.826531	NaN	-20.217338	-1.891416	-2554.5	33.0	1987-01-02 12:00:00

	death	pm10median	pm25median	o3median	so2median	time	tmpd	date
idx								
4	135	5.566456	NaN	-19.675671	6.139341	-2553.5	29.0	1987-01-03 12:00:00
5	126	NaN	NaN	-19.217338	2.278465	-2552.5	32.0	1987-01-04 12:00:00
...	...	...	...	...	...	...	...	...
5110	119	-9.123944	10.988615	-13.274541	1.673339	2552.5	12.0	2000-12-26 12:00:00
5111	125	-12.123944	5.288615	-11.179778	2.618337	2553.5	13.0	2000-12-27 12:00:00
5112	132	-3.123944	0.525000	-6.036201	4.931991	2554.5	21.0	2000-12-28 12:00:00
5113	133	-26.123944	-10.061385	-0.598701	-1.456316	2555.5	27.0	2000-12-29 12:00:00
5114	145	-25.123944	-1.811385	-4.890368	0.090006	2556.5	16.0	2000-12-30 12:00:00

Problem 2.1

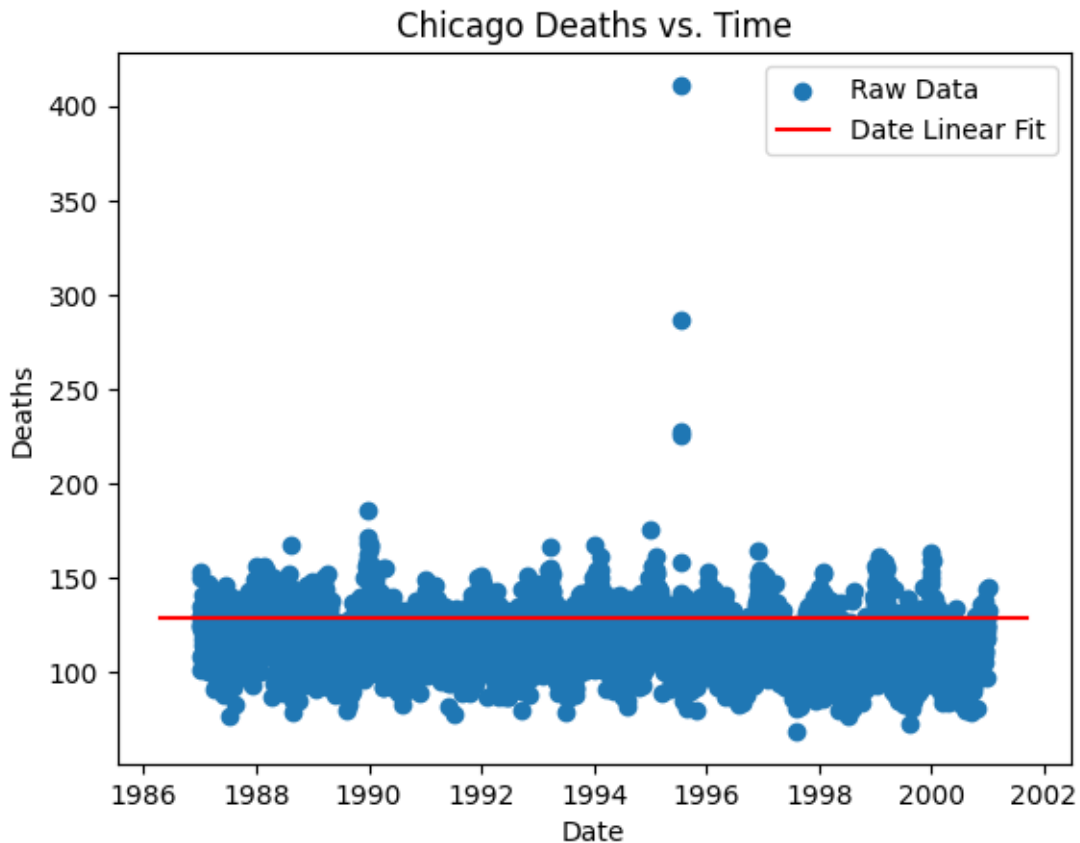
Load and plot the deaths data. Linearly regress the number of deaths against time and report the coefficients for the fitted model. Add the regression line to your plot. What does this regression tell you?

```
model = sklearn.linear_model.LinearRegression()
model.fit(pd.DataFrame(chi["date"]), pd.DataFrame(chi["death"]))
print(model.coef_[0][0], model.intercept_[0])
```

```
-1.7600693798433622e-17 128.7477852258744
```

```
plt.scatter(chi["date"], chi["death"], label="Raw Data")
plt.xlabel("Date")
plt.ylabel("Deaths")
plt.title("Chicago Deaths vs. Time")
x = np.linspace(plt.xlim()[0], plt.xlim()[1], 100)
plt.plot(x, model.predict(x.reshape(-1, 1)), "r-", label="Date Linear Fit")
plt.legend()
plt.savefig("figs/21.png")
```

```
/opt/anaconda3/envs/weather_plots/lib/python3.14/site-packages/sklearn/utils/validation.py:2
warnings.warn(
```



Problem 2.2

Now linearly regress the deaths data against temperature. Once again, report the coefficients, and add the regression line to your plot. What does this regression tell you about the influence of temperature?

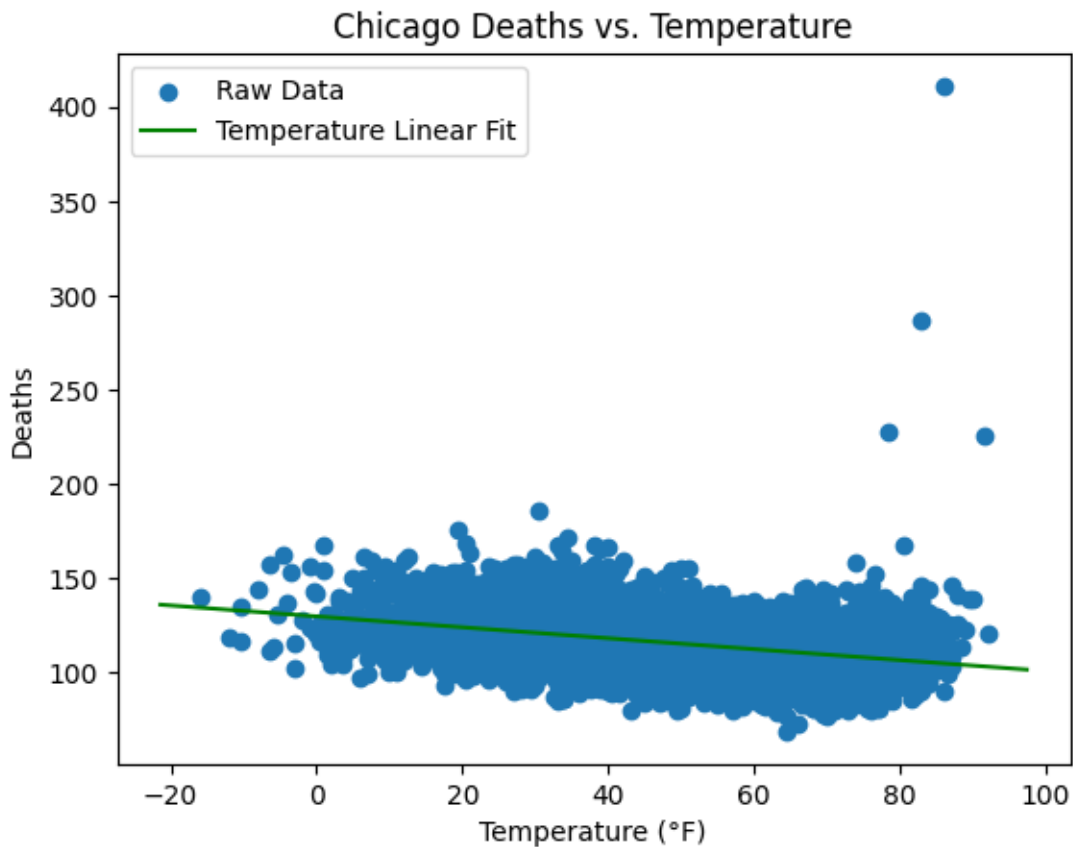
```
model = sklearn.linear_model.LinearRegression()
model.fit(pd.DataFrame(chi["tmpd"]), pd.DataFrame(chi["death"]))
print(model.coef_[0][0], model.intercept_[0])
```

```
-0.2896442961500986 129.9570512147682
```

```
plt.scatter(chi["tmpd"], chi["death"], label="Raw Data")
plt.xlabel("Temperature (°F)")
plt.ylabel("Deaths")
plt.title("Chicago Deaths vs. Temperature")
```

```
x = np.linspace(plt.xlim()[0], plt.xlim()[1], 100)
plt.plot(x, model.predict(x.reshape(-1,1)), "g-", label="Temperature Linear Fit")
plt.legend()
plt.savefig("figs/22.png")
```

```
/opt/anaconda3/envs/weather_plots/lib/python3.14/site-packages/sklearn/utils/validation.py:2
warnings.warn(
```



```
chi_cut = chi[['tmpd', "pm25median", "death"]].dropna()
```

```
model_new = sklearn.linear_model.LinearRegression()
model_new.fit(chi_cut[['tmpd', "pm25median"]], pd.DataFrame(chi_cut["death"]))
print(model_new.coef_[0], model_new.intercept_[0])
```

```
[-0.37111818  0.21532754] 129.29393325279526
```

```

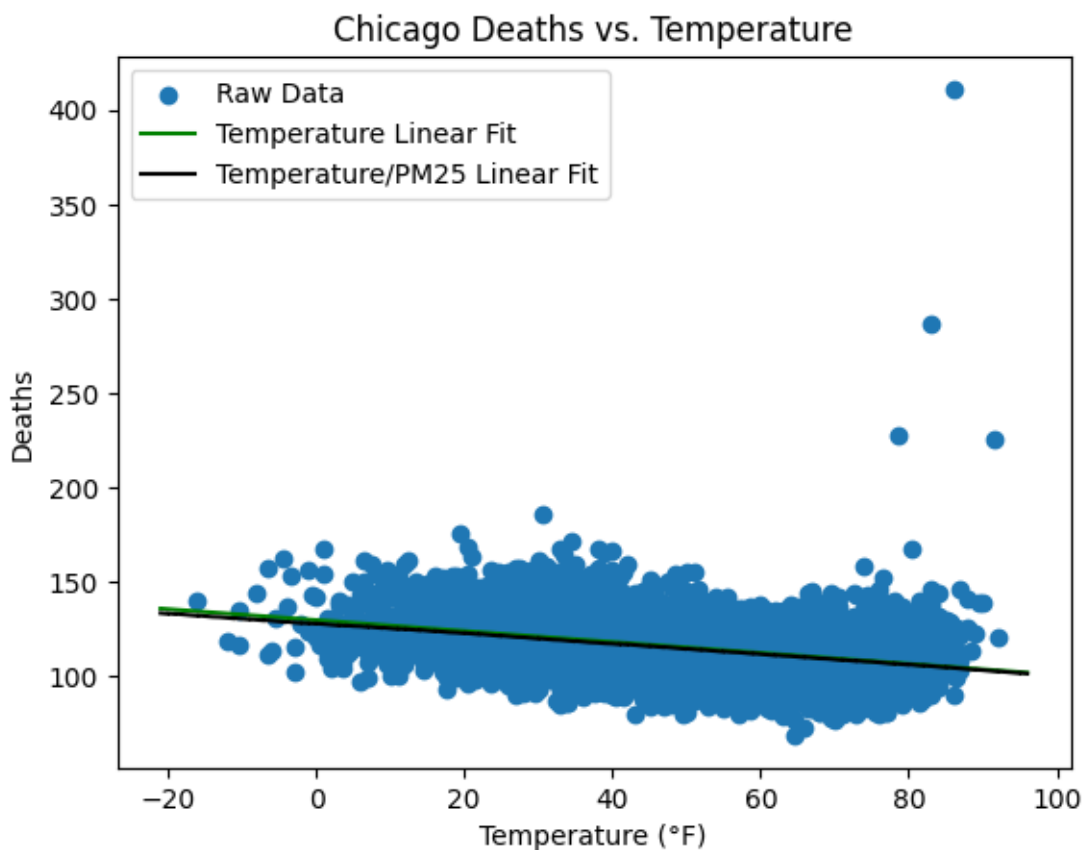
plt.scatter(chi["tmpd"], chi["death"], label="Raw Data")
plt.xlabel("Temperature (°F)")
plt.ylabel("Deaths")
plt.title("Chicago Deaths vs. Temperature")
x = np.sort(np.random.randint(plt.xlim()[0], plt.xlim()[1], 1000))
x2 = np.sort(np.random.randint(chi["pm25median"].min(), chi["pm25median"].max(), 1000))
plt.plot(x, model.predict(x.reshape(-1,1)), "g-", label="Temperature Linear Fit")
plt.plot(x, model_new.predict(np.stack([x, x2], axis=1)), "k-", label="Temperature/PM25 Linear Fit")
plt.legend()
plt.savefig("figs/23.png")

```

```

/opt/anaconda3/envs/weather_plots/lib/python3.14/site-packages/sklearn/utils/validation.py:2
warnings.warn(
/opt/anaconda3/envs/weather_plots/lib/python3.14/site-packages/sklearn/utils/validation.py:2
warnings.warn(

```



```
sklearn.model_selection.cross_val_score(sklearn.linear_model.LinearRegression(),
    pd.DataFrame(chi["tmpd"]),
    pd.DataFrame(chi["death"]),
    cv=5,
    scoring="neg_mean_squared_error")
```

```
array([-164.42797856, -170.08771888, -178.88082097, -321.36085279,
       -203.50699814])
```

```
sklearn.model_selection.cross_val_score(sklearn.linear_model.LinearRegression(),
    chi_cut[["tmpd", "pm25median"]],
    pd.DataFrame(chi_cut["death"]),
    cv=5,
    scoring="neg_mean_squared_error")
```

```
array([-140.53480444, -170.12784263, -183.30495633, -150.66900383,
       -122.85507981])
```

```
aic = sklearn.linear_model.LassoLarsIC(criterion='aic')
```

```
aic.fit(chi_cut[["tmpd", "pm25median"]],
    pd.DataFrame(chi_cut["death"]))
```

```
/opt/anaconda3/envs/weather_plots/lib/python3.14/site-packages/sklearn/utils/validation.py:1
    y = column_or_1d(y, warn=True)
```

criterion criterion: {'aic', 'bic'}, default='aic'

The type of criterion to use.

<code>fit_intercept</code> <code>fit_intercept:</code> bool, default=True	True
Whether to calculate the intercept for this model. If set to false, no intercept will be used in calculations (i.e. data is expected to be centered).	
<code>verbose</code> <code>verbose:</code> bool or int, default=False	False
Sets the verbosity amount.	
<code>precompute</code> <code>precompute:</code> bool, 'auto' or array-like, default='auto'	'auto'
Whether to use a precomputed Gram matrix to speed up calculations. If set to “auto” let us decide. The Gram matrix can also be passed as argument.	
<code>max_iter</code> <code>max_iter:</code> int, default=500	500
Maximum number of iterations to perform. Can be used for early stopping.	

eps	eps: float, default=np.finfo(float).eps	np.float64(2....049250313e-16)
-----	--	--------------------------------

The machine-precision regularization in the computation of the Cholesky diagonal factors. Increase this for very ill-conditioned systems. Unlike the “tol” parameter in some iterative optimization-based algorithms, this parameter does not control the tolerance of the optimization.

copy_X	copy_X: bool, default=True	True
--------	-------------------------------	------

If True, X will be copied; else, it may be overwritten.

positive: bool, default=False	False
----------------------------------	-------

Restrict coefficients to be ≥ 0 . Be aware that you might want to remove `fit_intercept` which is set `True` by default. Under the positive restriction the model coefficients do not converge to the ordinary-least-squares solution for small values of `alpha`. Only coefficients up to the smallest `alpha` value (`"alphas_[alphas_ > 0.].min()"` when `fit_path=True`) reached by the stepwise Lars-Lasso algorithm are typically in congruence with the solution of the coordinate descent Lasso estimator. As a consequence using `LassoLarsIC` only makes sense for problems where a sparse solution is expected and/or reached.

<code>noise_variance</code>	None
<code>noise_variance: float,</code>	
<code>default=None</code>	

The estimated noise variance of the data. If 'None', an unbiased estimate is computed by an OLS model. However, it is only possible in the case where 'n_samples > n_features + fit_intercept'.

.. versionadded:: 1.1
